Relatório Técnico
DevSecOps Avançado no ERP Escolar

Marcos Antônio Ferreira de Araújo
Matheus de Castro Viana

Mogi das Cruzes
2026

1. Introdução à Arquitetura Open-Source

Atingir um nível profissional de maturidade em segurança da informação (SecDevOps) não exige obrigatoriamente a adoção de plataformas comerciais. Utilizando uma arquitetura 100% gratuita e open-source integrada ao GitHub Actions (com runner ubuntu-24.04), implementou-se uma esteira automatizada de Continuous Integration (CI). Sob o conceito de Shift-Left Security, os testes foram deslocados para as fases iniciais do ciclo de desenvolvimento, sendo executados a cada push ou Pull Request.

O ERP Escolar é desenvolvido em Python 3.14.6, Django 6.1 e PostgreSQL 18.6. O ambiente da esteira constrói um contêiner Docker a partir da imagem base (python:3.14-slim), instala as dependências, executa as migrações em um banco temporário e sobe o servidor gunicorn em background na porta 8000. As chaves de segurança (SECRET_KEY, FIELD_ENCRYPTION_KEYS e BLIND_INDEX_KEY) são geradas de forma randômica exclusivamente para a execução, assegurando que segredos de produção nunca sejam expostos.

2. Descritivo Técnico das Ferramentas

A esteira automatizada orquestra três ferramentas principais de análise e varredura:

• **OWASP ZAP (DAST):** Ferramenta de Dynamic Application Security Testing executada a partir da imagem oficial ghcr.io/zaproxy/zaproxy:stable. O ZAP interage com a aplicação em execução no ambiente efêmero (http://127.0.0.1:8000) simulando ataques reais. Utiliza-se o script zap-full-scan.py, configurado para identificar vulnerabilidades como Cross-Site Scripting (XSS), injeções, e configurações incorretas de cabeçalhos. Os resultados são exportados em zap-report.json e a esteira implementa um gate que bloqueia o fluxo caso alertas com riskcode 3 (Risco Alto) sejam encontrados.

• **Bearer (Privacidade e SAST):** Responsável pela análise estática do código (Static Application Security Testing) focada em privacidade. Executa escâneres sast e secrets para buscar credenciais expostas no código e padrões inseguros. Gera os artefatos bearer-security.json e bearer-privacy.json, que mapeiam os tipos de dados pessoais e os fluxos por onde trafegam, sem a resolução de domínios externos (disable-domain-resolution). Interrompe a execução caso falhas de nível Crítico (Critical) sejam detectadas.

• **Trivy (SBOM e Cadeia de Suprimentos):** Scanner da Aqua Security focado em Supply Chain Security. O Trivy inspeciona a imagem Docker gerada e lista todas as dependências do sistema operacional e bibliotecas Python em um Software Bill of Materials (SBOM). O artefato school-erp-sbom.cdx.json é gerado no formato padronizado CycloneDX, mapeando versões fixadas de dependências como django-allauth, pyotp, cryptography e argon2-cffi, essencial para rastreamento de CVEs futuras.

3. Integração com os Requisitos do Projeto

A esteira implementada garante a conformidade e a segurança exigidas pelo documento de requisitos do sistema. O ERP Escolar manipula Informações Pessoalmente Identificáveis (PII) sensíveis de alunos e funcionários, exigindo proteção estrita de acordo com a LGPD (Lei Geral de Proteção de Dados). O Bearer assegura que nenhuma manipulação não mapeada de PII chegue à ramificação principal (main), confrontando os fluxos lógicos do código com o inventário de dados cadastrados.

Paralelamente, a geração automatizada do SBOM pelo Trivy assegura a total transparência da cadeia de suprimentos para stakeholders e auditores, permitindo um mapeamento preciso caso dependências vitais (como o PyOTP usado no 2FA ou a biblioteca cryptography 50.0.2 usada na cifração Fernet) apresentem vulnerabilidades zero-day.

As políticas de bloqueio baseadas no código de saída (exit-code: 1) do ZAP e do Bearer transformam a pipeline em um portão de segurança rigoroso, onde o build falha imediatamente perante ameaças críticas.

4. Evidências de Funcionamento

Abaixo seguem os registros de execução que comprovam a efetividade dos controles automatizados da esteira SecDevOps:

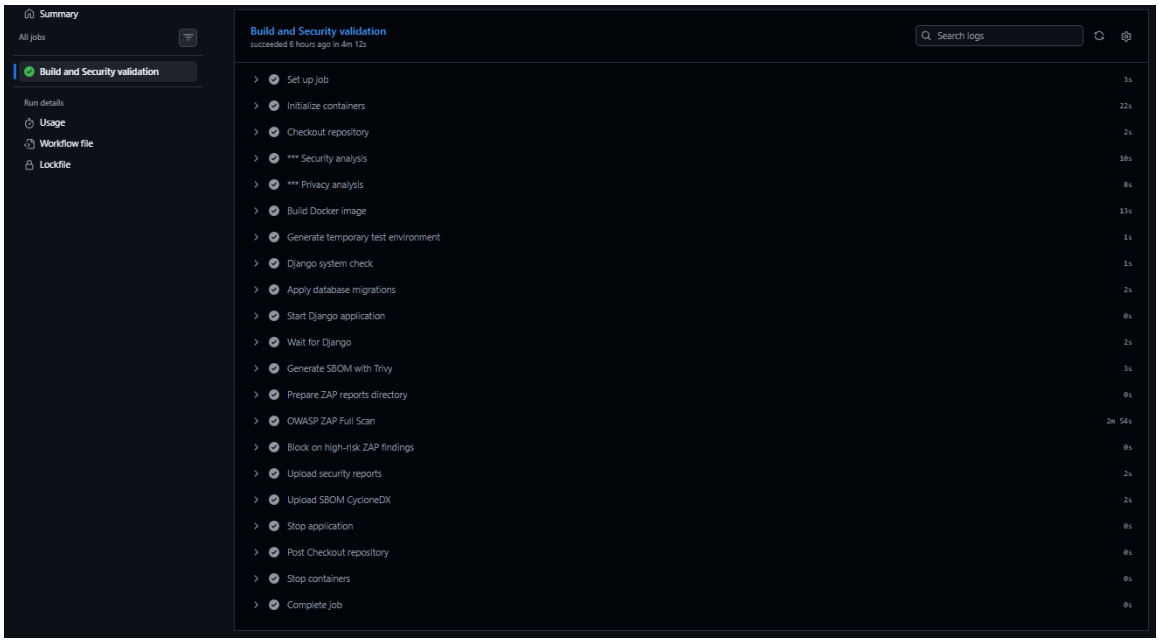


Figura 1 - Job Build and Secutity validation rodando

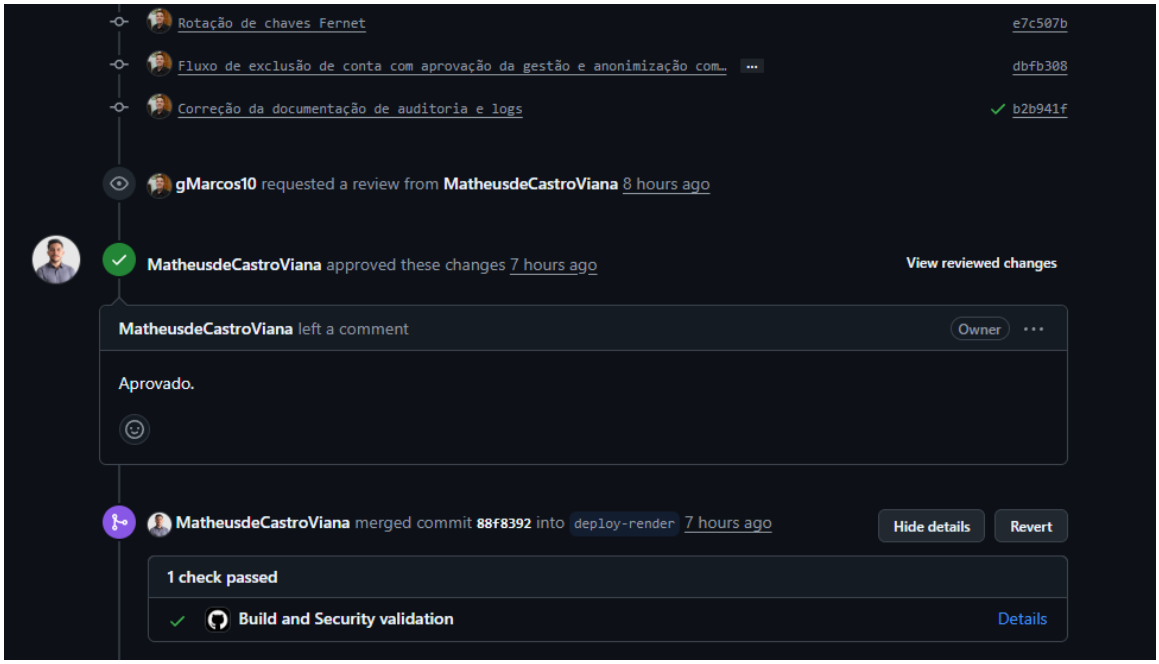


Figura 2 - Job rodando automaticamente após pull request

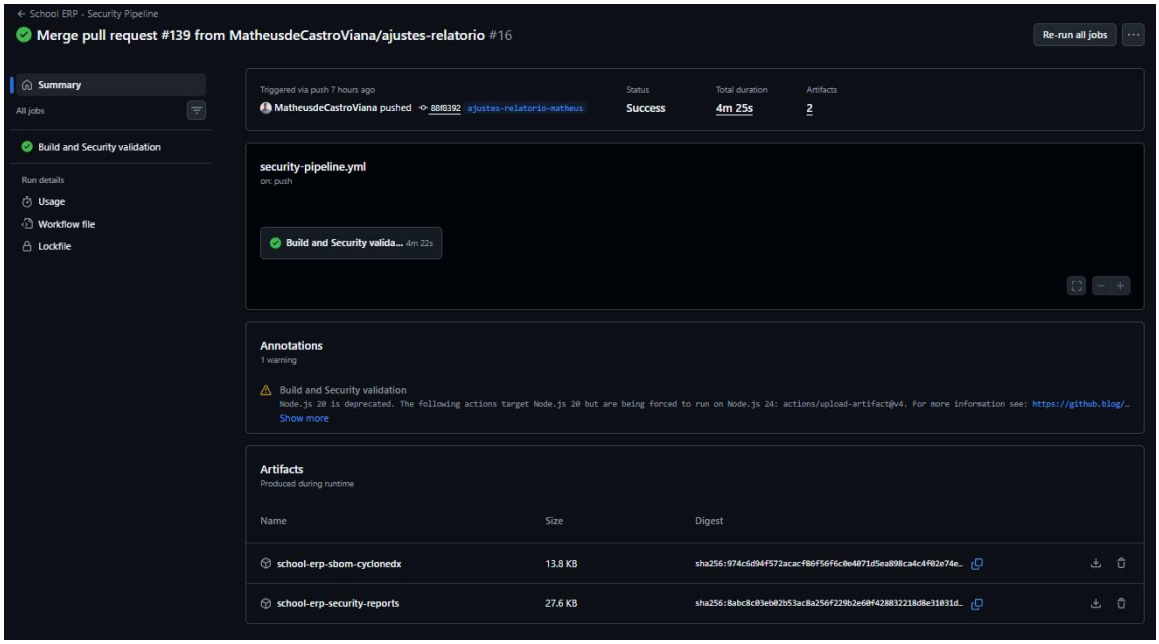


Figura 3 - Artefatos disponíveis para download

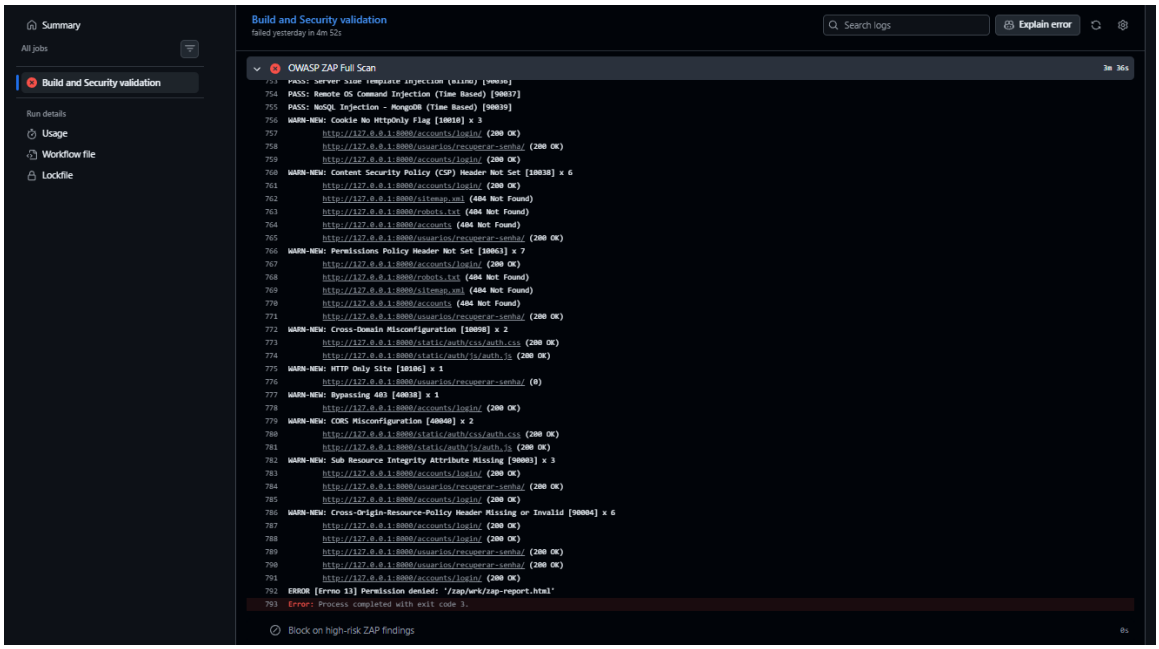


Figura 4 - Falha no OWASP ZAP Scan